

A · Chuleta de pandas para series temporales

Quant-Backtester · material de estudio

Esta chuleta reúne las herramientas de pandas que vas a necesitar en los TODO del proyecto. No es un capítulo: aquí no hay TODO, y los ejemplos son de juguete a propósito (temperaturas, ventas, lecturas de un sensor). Te enseñan cómo se comporta cada herramienta; la combinación que resuelve cada TODO la pones tú. Todo funciona igual en pandas 2.2 y en pandas 3.x salvo donde se indica.

Ten en cuenta una particularidad del proyecto: `pyproject.toml` convierte en error cualquier `FutureWarning` o `DeprecationWarning` emitido desde `src/`. Una API obsoleta que en pandas 2.2 "solo avisa" hace fallar los tests.

Todos los ejemplos parten de esto:

```
import numpy as np
import pandas as pd

dias = pd.to_datetime(["2024-03-04", "2024-03-05", "2024-03-06", "2024-03-07", "2024-03-08"])
temp = pd.Series([12.0, 15.0, 13.0, 17.0, 16.0], index=dias, name="temp")
```

DatetimeIndex y alineación por etiquetas

Una serie temporal en pandas es una `Series` o un `DataFrame` cuyo índice es un `DatetimeIndex`; si tus fechas son texto, `pd.to_datetime` las convierte. Las operaciones entre dos objetos se alinean por **etiqueta** (la fecha y, en un `DataFrame`, también la columna), no por posición: lo que no está en los dos lados sale NaN. Es una red de seguridad y una trampa a la vez: si las fechas no coinciden no hay error, solo NaN silenciosos.

```
lluvia = pd.Series([0.0, 2.5, 1.0], index=pd.to_datetime(["2024-03-05", "2024-03-06", "2024-03-11"]))
temp + lluvia
```

```
2024-03-04    NaN
2024-03-05    15.0
2024-03-06    15.5
2024-03-07     NaN
2024-03-08     NaN
2024-03-11     NaN
dtype: float64
```

Con `temp.add(lluvia, fill_value=0)` lo que falta en un lado cuenta como 0, y sale 12.0, 15.0, 15.5, 17.0, 16.0 y 1.0. Ojo con `DataFrame * Series`: pandas alinea el índice de la `Series` con las **columnas** del `DataFrame`. Para operar fecha a fecha usa `df.mul(s, axis=0)`.

Dónde lo usarás: TODO 2.4 (posiciones × rendimientos), TODO 2.5 y TODO 6.1 (posiciones × escala). El motor exige que precios y señales tengan exactamente las mismas fechas y columnas precisamente para que esta alineación no fabrique NaN.

shift: mover valores en el tiempo

`shift(n)` desplaza los **valores** n filas hacia fechas posteriores y conserva el índice: con n = 1, en cada fecha ves el valor de la fila anterior ("el de ayer"). Las n primeras filas quedan NaN, salvo que pases `fill_value`. Con n negativo traes valores del futuro, y en un backtest eso casi siempre es look-ahead. `shift` cuenta filas, no días de calendario: en una tabla de fines de mes, `shift(2)` significa "hace dos meses".

```
pd.DataFrame({
    "temp": temp,
    "ayer": temp.shift(1),
    "mañana": temp.shift(-1),
    "ayer_o_0": temp.shift(1, fill_value=0.0),
})
```

	temp	ayer	mañana	ayer_o_0
2024-03-04	12.0	NaN	15.0	0.0
2024-03-05	15.0	12.0	13.0	12.0
2024-03-06	13.0	15.0	17.0	15.0
2024-03-07	17.0	13.0	16.0	13.0
2024-03-08	16.0	17.0	NaN	17.0

```
fines_de_mes = pd.to_datetime(["2024-01-31", "2024-02-29", "2024-03-28", "2024-04-30"])
ventas_mes = pd.Series([100.0, 120.0, 90.0, 130.0], index=fines_de_mes)
ventas_mes.shift(2)
```

2024-01-31	NaN
2024-02-29	NaN
2024-03-28	100.0
2024-04-30	120.0

dtype: float64

Dónde lo usarás: TODO 1.1 (el precio de ayer), TODO 2.1 (de señales a posiciones), TODO 2.2 (la posición anterior), TODO 3.2 (desplazarse meses en la tabla mensual) y TODO 6.1 (la volatilidad de ayer).

diff: diferencia con la fila anterior

`diff(n)` es `x - x.shift(n)`: la variación respecto a n filas antes. Las n primeras filas quedan NaN porque no hay con qué comparar. Qué valor deberían tener depende del problema, y `diff` no puede saberlo: si lo necesitas, decídelo tú.

```
pd.DataFrame({"temp": temp, "diff1": temp.diff(), "diff2": temp.diff(2)})
```

	temp	diff1	diff2
2024-03-04	12.0	NaN	NaN
2024-03-05	15.0	3.0	NaN
2024-03-06	13.0	-2.0	1.0
2024-03-07	17.0	4.0	2.0
2024-03-08	16.0	-1.0	3.0

Dónde lo usarás: TODO 2.2 (`compute_turnover`), cuyo docstring te dice qué debe valer la primera fila.

pct_change y el fill_method obsoleto

`pct_change()` calcula $x_t / x_{t-1} - 1$. Hasta pandas 2.x, por defecto rellenaba los huecos internos con el último valor antes de calcular (`fill_method="pad"`); desde la 2.1 eso está obsoleto y, si hay NaN internos, avisa con `FutureWarning`. En pandas 3 ya no rellena (el único valor admitido de `fill_method` es `None`). Resultado: el mismo código da números distintos según la versión.

```
ventas = pd.Series([100.0, 110.0, np.nan, 121.0], index=dias[:4])
ventas.pct_change()
```

pandas 2.2 (rellena con 110 y avisa)		pandas 3 (no rellena)	
2024-03-04	NaN	2024-03-04	NaN
2024-03-05	0.1	2024-03-05	0.1
2024-03-06	0.0	2024-03-06	NaN
2024-03-07	0.1	2024-03-07	NaN
dtype: float64		dtype: float64	

Fíjate en lo que inventa pandas 2.2: un cambio del 0 % en un día sin dato y un 10 % el día 7 calculado contra un valor que no existía. Si usas `pct_change`, pasa `fill_method=None` para que se comporte igual en las dos versiones. En el proyecto, el docstring del TODO 1.1 te pide escribir el cociente tú mismo con `shift`, que deja explícito qué pasa con los huecos.

Dónde lo usarás: TODO 1.1 (para entender por qué su docstring desaconseja `pct_change`).

rolling: ventanas móviles

`rolling(window)` agrupa, para cada fila, las `window` filas que terminan en ella, **incluida**, y aplica una reducción: `mean`, `std`, `sum`, `max`... `min_periods` es el número mínimo de valores no NaN para dar resultado; por defecto es igual a `window`, así que las primeras `window - 1` filas salen NaN. La `std` de pandas usa `ddof = 1` (muestral) por defecto; `np.std` usa `ddof = 0` (poblacional). Sobre un DataFrame, `rolling` trabaja columna a columna.

```
ventas5 = pd.Series([10.0, 12.0, 11.0, 15.0, 14.0], index=dias)
pd.DataFrame({
    "x": ventas5,
    "media3": ventas5.rolling(3).mean(),
    "media3_min1": ventas5.rolling(3, min_periods=1).mean(),
    "std3": ventas5.rolling(3).std(),
}).round(2)
```

	x	media3	media3_min1	std3
2024-03-04	10.0	NaN	10.00	NaN
2024-03-05	12.0	NaN	11.00	NaN
2024-03-06	11.0	11.00	11.00	1.00
2024-03-07	15.0	12.67	12.67	2.08
2024-03-08	14.0	13.33	13.33	2.08

```
print(np.std([10.0, 12.0, 11.0])) # numpy: ddof=0 por defecto
print(ventas5.iloc[:3].std())     # pandas: ddof=1 por defecto
```

```
0.816496580927726
1.0
```

Dónde lo usarás: TODO 3.1 (medias rápida y lenta), TODO 3.3 (media y σ de las bandas) y TODO 6.1 (volatilidad móvil). La diferencia de ddof importa también en los TODO 4.1 y 4.3 si trabajas con arrays de numpy.

expanding y los acumulados

`expanding()` es una ventana que empieza siempre en la primera fila y va creciendo: `expanding().mean()` es la media "hasta hoy". Para los casos más comunes hay atajos: `cumsum()`, `cumprod()`, `cummax()` y `cummin()`. Los acumulados saltan los NaN: el NaN se queda en su sitio y el acumulado continúa después. `sum()` y `prod()`, sin "cum", dan solo el total.

```
pd.DataFrame({
    "temp": temp,
    "media_hasta_hoy": temp.expanding().mean(),
    "cummax": temp.cummax(),
    "cumsum": temp.cumsum(),
}).round(2)
```

	temp	media_hasta_hoy	cummax	cumsum
2024-03-04	12.0	12.00	12.0	12.0
2024-03-05	15.0	13.50	15.0	27.0
2024-03-06	13.0	13.33	15.0	40.0
2024-03-07	17.0	14.25	17.0	57.0
2024-03-08	16.0	14.60	17.0	73.0

```
factores = pd.Series([2.0, 3.0, 0.5, 4.0], index=dias[:4])
factores.cumprod()
```

```
2024-03-04    2.0
2024-03-05    6.0
2024-03-06    3.0
2024-03-07   12.0
dtype: float64
```

Dónde lo usarás: TODO 2.6 (producto acumulado), TODO 4.2 (producto total) y TODO 4.5 (máximo acumulado). `cumulative_turnover` de `BacktestResult`, ya hecho, es un `cumsum`.

ffill(limit=...) y por qué no bfill

`ffill()` rellena cada NaN con el último valor conocido anterior; `limit=n` rellena como mucho n NaN seguidos de cada hueco y deja el resto en NaN. Los NaN iniciales no se rellenan, porque no hay valor anterior. `bfill()` rellena con el siguiente valor conocido, es decir, copia el futuro en el pasado: en un backtest es look-ahead. `fillna(method="ffill")` está obsoleto en 2.x y eliminado en pandas 3: usa `.ffill()`. `fillna(0.0)` sigue siendo la forma de poner un valor fijo.

```
sensor = pd.Series([20.0, np.nan, np.nan, np.nan, 25.0], index=dias)
pd.DataFrame({
    "sensor": sensor,
    "ffill": sensor.ffill(),
    "ffill_lim2": sensor.ffill(limit=2),
    "bfill": sensor.bfill(),
})
```

	sensor	ffill	ffill_lim2	bfill
2024-03-04	20.0	20.0	20.0	20.0
2024-03-05	NaN	20.0	20.0	25.0
2024-03-06	NaN	20.0	20.0	25.0
2024-03-07	NaN	20.0	NaN	25.0
2024-03-08	25.0	25.0	25.0	25.0

La columna `bfill` "sabe" el 5 de marzo que el 8 habrá 25 grados.

Dónde lo usarás: TODO 1.3 (paso 4 de `clean_prices`), TODO 3.2 (estirar la señal de fin de mes a todos los días) y TODO 3.3 (propagar el estado dentro/fuera).

mask y where

`mask(cond, otro)` sustituye por `otro` (NaN si no lo das) los valores donde `cond` es True.

`where(cond, otro)` hace lo contrario: **conserva** donde `cond` es True y sustituye el resto. Son una el espejo de la otra. Ambas mantienen índice y columnas, y `cond` puede ser una Series o un DataFrame booleano con la misma forma.

```
lectura = pd.Series([12.0, -999.0, 13.0, 17.0, -999.0], index=dias)
pd.DataFrame({
    "lectura": lectura,
    "mask": lectura.mask(lectura < -50),
    "where": lectura.where(lectura > -50, 0.0),
})
```

	lectura	mask	where
2024-03-04	12.0	12.0	12.0
2024-03-05	-999.0	NaN	0.0
2024-03-06	13.0	13.0	13.0
2024-03-07	17.0	17.0	17.0
2024-03-08	-999.0	NaN	0.0

Dónde lo usarás: TODO 1.3 (paso 2: marcar como NaN los valores imposibles) y, si te encaja, TODO 3.3.

dropna(how=...)

Sobre un DataFrame, `dropna()` elimina filas con NaN. Con `how="any"` (por defecto) basta con que **algún** valor de la fila sea NaN; con `how="all"`, solo si lo son **todos**. `subset=[...]` limita las columnas que se miran y `axis=1` elimina columnas en vez de filas. Relacionado: `serie.first_valid_index()` devuelve la etiqueta del primer valor no NaN, útil para saber dónde empieza cada columna.

```
tabla = pd.DataFrame(
    {"Madrid": [12.0, np.nan, 13.0, np.nan], "Bilbao": [9.0, np.nan, np.nan, 11.0]},
    index=dias[:4],
)
tabla.dropna(how="all")
```

	Madrid	Bilbao
2024-03-04	12.0	9.0
2024-03-06	13.0	NaN
2024-03-07	NaN	11.0

```
tabla.dropna() # how="any"
```

	Madrid	Bilbao
2024-03-04	12.0	9.0

Dónde lo usarás: TODO 1.3 (pasos 3 y 5 de `clean_prices`).

`uplicated(keep=...)`

`uplicated` marca con `True` las repeticiones. Con `keep="first"` (por defecto) no marca la primera aparición; con `keep="last"`, no marca la última; con `keep=False`, las marca todas. Cuidado con a qué se lo aplicas: `df.duplicated()` compara los **valores** de filas enteras, no las fechas; para fechas repetidas usa `df.index.duplicated(...)`. Para quedarte con lo no marcado, selecciona con la máscara negada (`~`).

```
idx = pd.to_datetime(["2024-03-04", "2024-03-05", "2024-03-05", "2024-03-06"])
print(idx.duplicated(keep="first"))
print(idx.duplicated(keep="last"))
print(idx.duplicated(keep=False))
```

```
[False False  True False]
[False  True False False]
[False  True  True False]
```

Dónde lo usarás: TODO 1.3 (paso 1 de `clean_prices`).

`resample, to_period("M")` y el alias `"ME"`

`resample("ME")` agrupa por meses de calendario y **etiqueta cada grupo con el último día natural del mes**, esté o no en tus datos; además, crea filas para los meses sin datos. `index.to_period("M")` convierte cada fecha en su mes (un `Period`) sin inventar nada: si agrupas por él, obtienes solo los meses con datos. Desde pandas 2.2, el alias de fin de mes para offsets, `resample` y `date_range` es `"ME"`; `"M"` avisa en 2.2 y da error en pandas 3. Para **periodos** sigue siendo `"M"`, y `to_period("ME")` es un error.

```
v = pd.Series(
    [5.0, 7.0, 3.0, 4.0, 6.0],
    index=pd.to_datetime(["2024-01-30", "2024-01-31", "2024-02-01", "2024-02-02", "2024-04-02"]),
)
v.resample("ME").last()
```

2024-01-31	7.0
2024-02-29	4.0
2024-03-31	NaN
2024-04-30	6.0

Freq: ME, dtype: float64

```
v.index.to_period("M")
v.groupby(v.index.to_period("M")).last()
```

```
PeriodIndex(['2024-01', '2024-01', '2024-02', '2024-02', '2024-04'], dtype='period[M]')

2024-01    7.0
2024-02    4.0
2024-04    6.0
Freq: M, dtype: float64
```

La etiqueta 2024-02-29 no está en los datos (el último dato de febrero es del día 2) y marzo aparece con NaN; con `sum()` en vez de `last()`, ese mes vacío valdría 0. En un backtest, una etiqueta que no es día de mercado no sirve para volver al índice diario.

Dónde lo usarás: TODO 3.2. La función `_month_end_dates`, ya hecha, usa `to_period("M")` para encontrar el último día **con datos** de cada mes, justo para no depender de las etiquetas de `resample`.

reindex

`reindex(nuevo_indice)` devuelve el objeto con exactamente las etiquetas pedidas: las que ya existían conservan su valor, las nuevas salen NaN (o `fill_value`) y las que no están en el nuevo índice desaparecen. No interpola ni busca la fecha más cercana. Si el índice de partida tiene etiquetas repetidas, lanza `ValueError` ("cannot reindex on an axis with duplicate labels").

```
puntual = pd.Series([1.0, 3.0], index=pd.to_datetime(["2024-03-04", "2024-03-07"]))
pd.DataFrame({
    "reindex": puntual.reindex(dias),
    "con_0": puntual.reindex(dias, fill_value=0.0),
    "con_ffill": puntual.reindex(dias).ffill(),
})
```

	reindex	con_0	con_ffill
2024-03-04	1.0	1.0	1.0
2024-03-05	NaN	0.0	1.0
2024-03-06	NaN	0.0	1.0
2024-03-07	3.0	3.0	3.0
2024-03-08	NaN	0.0	3.0

Dónde lo usarás: TODO 3.2 (de la tabla mensual al índice diario) y TODO 5.4 (las posiciones de los tramos de test sobre el índice completo).

.loc frente a .iloc y máscaras booleanas

`.loc` selecciona por **etiqueta** y, al cortar con fechas, incluye **los dos** extremos. `.iloc` selecciona por **posición**, como las listas de Python: el final queda fuera. Para un intervalo semiabierto [a, b) de fechas, usa una máscara booleana que combine dos comparaciones del índice con `&`, cada una entre paréntesis. Y no uses `s[0]` para leer el primer valor de una serie con fechas: era una lectura posicional implícita que en pandas 3 da `KeyError`; usa `s.iloc[0]`.

```
temp.loc["2024-03-05":"2024-03-07"]    # etiquetas: los dos extremos entran
```

```
2024-03-05    15.0
2024-03-06    13.0
2024-03-07    17.0
Name: temp, dtype: float64
```

```
temp.iloc[1:3]                                # posiciones 1 y 2: la 3 no entra
a, b = pd.Timestamp("2024-03-05"), pd.Timestamp("2024-03-07")
temp[(temp.index >= a) & (temp.index < b)]    # [a, b): mismo resultado
```

```
2024-03-05    15.0
2024-03-06    13.0
Name: temp, dtype: float64
```

`.loc[:fecha]` incluye `fecha`, y `.loc[fecha:]` también: si los usas como "antes" y "después", esa fecha aparece en los dos.

Dónde lo usarás: TODO 4.6 (antes y después del valle), TODO 5.2 (la historia anterior a `end` y el tramo `[start, end)`) y TODO 5.4. `Window.train_mask/test_mask` y `BacktestResult.slice`, ya hechos, son respectivamente máscaras semiabiertas y un `.loc` con los dos extremos.

DateOffset frente a Timedelta

`pd.Timedelta(days=365)` es una duración fija; `pd.DateOffset(years=1)` es una regla de calendario ("el mismo día, un año después"). Con años bisiestos no coinciden. Si el día resultante no existe, `DateOffset` lo ajusta al último día del mes. Cuidado con el singular: `DateOffset(year=2020)` no suma un año, **fija** el año a 2020.

```
t = pd.Timestamp("2023-03-01")
print(t + pd.Timedelta(days=365))
print(t + pd.DateOffset(years=1))
print(pd.Timestamp("2024-02-29") + pd.DateOffset(years=1))
print(pd.Timestamp("2024-01-31") + pd.DateOffset(months=1))
```

```
2024-02-29 00:00:00
2024-03-01 00:00:00
2025-02-28 00:00:00
2024-02-29 00:00:00
```

Dónde lo usarás: TODO 5.1 (inicios y fines de las ventanas en años de calendario).

idxmin e idxmax

`idxmax()` devuelve la **etiqueta** (aquí, la fecha) del máximo; `argmax()` devuelve su **posición**. Con empates, la primera aparición. Los NaN se ignoran; si todo es NaN, pandas 3 lanza `ValueError` (2.x avisaba). En un DataFrame dan un resultado por columna.

```
print(temp.idxmax())
print(temp.idxmin())
print(temp.argmax())
```

```
2024-03-07 00:00:00
2024-03-04 00:00:00
3
```

Dónde lo usarás: TODO 4.6 (la fecha del valle del drawdown).

Cambios de pandas 3 que te afectan

pandas 3 activa por defecto *Copy-on-Write*: cualquier objeto que sacas de otro (una columna, un filtro) se comporta como una copia independiente. La consecuencia práctica es que la asignación encadenada `df["a"][mascara] = x` ya no modifica `df`: pandas 3 emite un aviso `ChainedAssignmentError` y no toca nada, mientras que pandas 2.2 todavía la ejecutaba (en muchos casos, con un aviso de que dejaría de funcionar). Escribe siempre en un solo paso con `.loc[filas, columna]`. Lo mismo vale para `df["a"].fillna(0, inplace=True)`: en pandas 3 no modifica `df`.

```
df = pd.DataFrame({"a": [1.0, 2.0, 3.0], "b": [10.0, 20.0, 30.0]})
df["a"][df["b"] > 15] = 0.0      # pandas 3: df NO cambia
df.loc[df["b"] > 15, "a"] = 0.0  # correcto en 2.2 y en 3
df
```

```
   a    b
0  1.0  10.0
1  0.0  20.0
2  0.0  30.0
```

El texto tiene ahora su propio dtype por defecto, `str`, en lugar de `object`. Afecta a comprobaciones del tipo `dtype == object`; para saber si una columna es de texto, usa `pd.api.types.is_string_dtype`, que funciona en las dos versiones.

```
pd.Series(["SPY", "QQQ"])
```

```
0    SPY
1    QQQ
dtype: str
```

En pandas 2.2, la última línea sería `dtype: object`. Resumen de los cambios que tocan este proyecto:

Patrón	pandas 2.2	pandas 3	Qué usar
<code>df["a"][m] = x</code>	funciona (puede avisar)	no modifica <code>df</code>	<code>df.loc[m, "a"] = x</code>
<code>pct_change()</code> con NaN internos	rellena y avisa	no rellena	<code>fill_method=None</code> o <code>shift</code>
<code>fillna(method="ffill")</code>	obsoleto, avisa	eliminado	<code>.ffill()</code>
<code>resample("M")</code>	avisa	error	<code>resample("ME")</code>
<code>to_period("M")</code>	correcto	correcto	sin cambios
<code>s[0]</code> con índice de fechas	avisa	<code>KeyError</code>	<code>s.iloc[0]</code>
<code>idxmax()</code> con todo NaN	avisa	<code>ValueError</code>	comprueba antes
columnas de texto	dtype <code>object</code>	dtype <code>str</code>	<code>is_string_dtype</code>

Dónde lo usarás: en todos los TODO. Como el proyecto convierte en error los avisos de obsolescencia que salen de `src/`, en pandas 2.2 la columna "avisa" equivale a un test en

rojo.